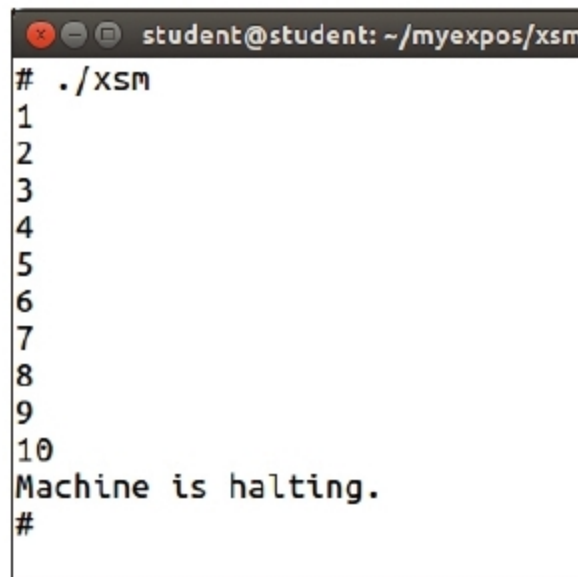


of the program *count.spl*. Notice that SPL is the programming language used to develop eXpOS.

Finally, let us consider an EXPL program called *prime.expl*.

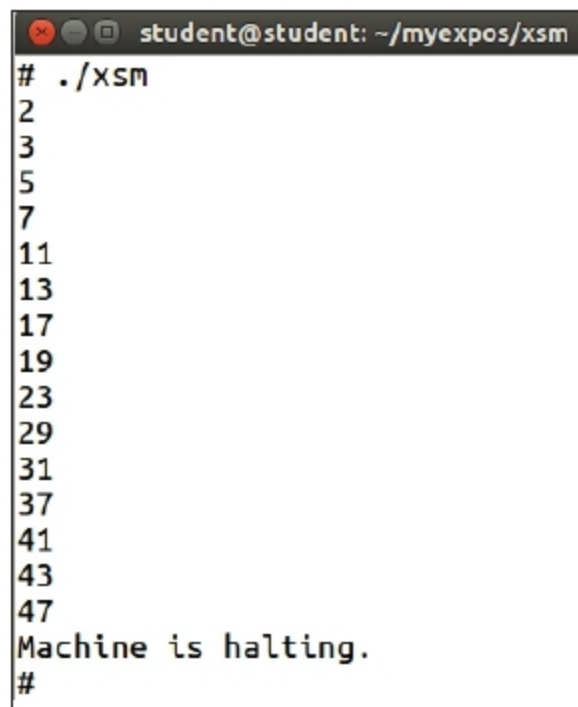
```
int main()
{
  decl
    int num, fl, ct, temp;
  enddecl
  begin
    num=2;
    while ( num <= 50 ) do
      fl=0;
      ct=2;
      while (ct < num) do
        if ((num % ct) == 0) then
          fl=1;
        break;
        endif;
        ct = ct + 1;
      endwhile;
      if (fl == 0) then
        temp = exposcall ( "Write" ,
-2, num );
        endif;
        num = num + 1;
      endwhile;
      return 0;
    end
  }
```

This program, which prints the prime numbers between 1 and 50, can be executed after developing eXpOS up to Stage 11. By now, you will have an OS start-up code capable of handling console output, a primitive exception handler and a simple exit system call implementation. Notice that all these modules will be further developed in later stages. As mentioned earlier, the XSM machine can only execute XSM assembly code. So, we use the EXPL compiler to compile the program *prime.expl* to obtain the XSM assembly language program *prime.xsm*. Figure 4 shows the output of the program *prime.expl* when executed.



```
student@student: ~/myexpos/xsm
# ./xsm
1
2
3
4
5
6
7
8
9
10
Machine is halting.
#
```

Figure 3: Output of the program *count.spl*



```
student@student: ~/myexpos/xsm
# ./xsm
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
Machine is halting.
#
```

Figure 4: Output of the program *prime.expl*

So far, we have only seen the execution of three programs on top of the built-in XSM machine. But in the later stages, we will add more modules to eXpOS so that more complicated application programs can be developed and executed on eXpOS. The sample EXPL programs provided by the development team include programs for bubble sort, quick sort, binary search tree creation, linked list creation, etc.

Now let us discuss a bit about the modules in a fully developed eXpOS

after all the 28 stages. Modules in eXpOS are used to perform certain logical tasks. They cannot be invoked from user programs. They are executed only in kernel mode. There will be an OS start-up code and boot module to load operating system data structures. The scheduler module is used for context switching between multiple processes. Device manager module and resource manager module are responsible for handling resources like the terminal, disk, etc. eXpOS also has a process manager module, timer module, pager module and memory manager module — just like any other typical operating system. It also has modules for exception handling and interrupt handling.

What I have covered in the article is the absolute minimum, but I urge students, young IT professionals and teachers of computer science to go through the detailed roadmap and explore eXpOS further. Completing all the 28 stages for the development of eXpOS might require some patience but I am sure that, in the end, you will be rewarded for your efforts. I also urge you to write far more complicated SPL and EXPL programs to test the limitations and vulnerabilities of eXpOS. Apart from NIT-C, a few other premier institutes in India have already shown interest in making eXpOS a part of their curriculum. I believe more and more institutes will join the team, and the use of eXpOS in Indian academia will rapidly increase in the coming years. I have used material from the roadmap for eXpOS for writing this article, and I thank the development team of the OS. **END** 🐧

By: Deepu Benson

The author is a free software enthusiast and his area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. He maintains a technical blog at www.computingforbeginners.blogspot.in.